

Actividad 3 — Consultas avanzadas en MongoDB: Aggregation Framework

Datos generales

- **Duración:** 50 minutos
- **Tipo:** Individual
- **Herramienta de IA:** Libre (OpenCode, ChatGPT, Claude, Gemini, Copilot...)
- **Requisitos:** MongoDB en Docker (Actividad 2)

Presupuesto de tiempo

Paso	Tiempo
Leer el marco teórico	5 min
Cargar los datos de ventas	8 min
Pipeline con \$match, \$group, \$sort	12 min
\$unwind, \$project y cálculos	10 min
\$lookup (equivalente a JOIN)	8 min
Verificación y entrega	7 min

Objetivos

1. Comprender el concepto de **pipeline de agregación**.
2. Usar las etapas \$match, \$group, \$sort, \$limit, \$project, \$unwind.
3. Hacer cálculos de agregación: sum, avg, count, max, min.
4. Usar \$lookup para relacionar dos colecciones.

Marco teórico

¿Qué es el Aggregation Framework?

MongoDB no usa SQL, pero ofrece un motor de **agregaciones** para analizar datos. Una agregación es un **pipeline**: un arreglo de etapas que se ejecutan en orden. Cada etapa recibe los documentos de la anterior, los transforma y los pasa a la siguiente.

```
db.ventas.aggregate([
  { $match: { ... } },    // 1. Filtrar
  { $unwind: "...ventas" }, // 2. Aplanar arreglos
  { $group: { _id: "...", total: { $sum: "$..." } } }, // 3. Agrupar y calcular
  { $sort: { total: -1 } } // 4. Ordenar
])
```

Etapas principales

Etapas	Función	Equivalente SQL
\$match	Filtrar documentos	WHERE
\$group	Agrupar y calcular	GROUP BY + agregados
\$project	Elegir/crear campos	SELECT
\$sort	Ordenar	ORDER BY
\$limit	Limitar resultados	LIMIT
\$unwind	Desarmar un arreglo en documentos	(desnormalizar filas)
\$lookup	Unir colecciones	JOIN
\$addFields	Agregar campos calculados	expresión

Operadores de acumulación

- \$sum → suma
- \$avg → promedio
- \$min / \$max → mínimo / máximo
- \$first / \$last → primero / último del grupo
- \$count (como etapa) → contar

Nota sobre \$group: el campo `_id` define el criterio de agrupación. Si querés agrupar todos los documentos juntos, usá `_id: null`.

Paso a paso

Paso 1 — Levantar MongoDB (si no sigue corriendo)

```
docker start mongo-clase
docker exec -it mongo-clase mongosh
```

Paso 2 — Crear base de datos de ventas

```
use comercio
```

Paso 3 — Insertar clientes

```
db.clientes.insertMany([
  { _id: 1, nombre: "Ana", ciudad: "Montevideo" },
  { _id: 2, nombre: "Bruno", ciudad: "Salto" },
  { _id: 3, nombre: "Carla", ciudad: "Montevideo" },
  { _id: 4, nombre: "Diego", ciudad: "Paysandú" }
])
```

Paso 4 — Insertar pedidos con detalle en arreglo

```
db.pedidos.insertMany([
  { cliente_id: 1, fecha: "2026-01-05", items: [
```

```

        { producto: "Teclado", cantidad: 1, precio: 40 },
        { producto: "Mouse", cantidad: 2, precio: 20 } ] ] },
    { cliente_id: 2, fecha: "2026-01-08", items: [
        { producto: "Monitor", cantidad: 1, precio: 250 } ] ] },
    { cliente_id: 1, fecha: "2026-02-01", items: [
        { producto: "Monitor", cantidad: 1, precio: 250 },
        { producto: "Webcam", cantidad: 1, precio: 60 } ] ] },
    { cliente_id: 3, fecha: "2026-02-10", items: [
        { producto: "Teclado", cantidad: 3, precio: 40 } ] ] },
    { cliente_id: 4, fecha: "2026-03-15", items: [
        { producto: "Mouse", cantidad: 5, precio: 20 } ] ] }
  ])
})

```

Paso 5 — Filtrar con \$match

```

db.pedidos.aggregate([
  { $match: { fecha: { $gte: "2026-02-01" } } }
])

```

Paso 6 — Desarmar el arreglo con \$unwind

\$unwind crea **un documento por cada elemento del arreglo** items:

```

db.pedidos.aggregate([
  { $unwind: "$items" }
])

```

Paso 7 — Calcular el total por pedido

Usamos \$project para crear un campo subtotal en cada ítem desarmado:

```

db.pedidos.aggregate([
  { $unwind: "$items" },
  { $project: {
    cliente_id: 1,
    fecha: 1,
    subtotal: { $multiply: ["$items.cantidad", "$items.precio"] }
  } },
  { $group: {
    _id: "$cliente_id",
    total_gastado: { $sum: "$subtotal" }
  } },
  { $sort: { total_gastado: -1 } }
])

```

Resultado esperado: Ana \$420, Bruno \$250, Carla \$120, Diego \$100.

Paso 8 — Productos más vendidos (por cantidad)

```

db.pedidos.aggregate([
  { $unwind: "$items" },
  { $group: {

```

```

        _id: "$items.producto",
        unidades: { $sum: "$items.cantidad" },
        ingresos: { $sum: { $multiply: ["$items.cantidad", "$items.precio"] } }
    } }
    } },
    { $sort: { unidades: -1 } }
])

```

Paso 9 — Estadísticas por mes

```

db.pedidos.aggregate([
  { $unwind: "$items" },
  { $group: {
    _id: { $substr: ["$fecha", 0, 7] },
    ventas: { $sum: { $multiply: ["$items.cantidad", "$items.precio"] } }
  } },
  { $group: {
    promedio_item: { $avg: "$items.precio" }
  } },
  { $sort: { _id: 1 } }
])

```

Paso 10 — Unir colecciones con \$lookup

Vamos a obtener los pedidos junto con el nombre del cliente (equivalente a un JOIN):

```

db.pedidos.aggregate([
  { $lookup: {
    from: "clientes",
    localField: "cliente_id",
    foreignField: "_id",
    as: "cliente"
  } },
  { $unwind: "$cliente" },
  { $project: {
    _id: 0,
    cliente_nombre: "$cliente.nombre",
    ciudad: "$cliente.ciudad",
    total_items: { $size: "$items" }
  } }
])

```

Paso 11 — Resumen general (todos juntos)

```

db.pedidos.aggregate([
  { $match: { fecha: { $gte: "2026-01-01" } } },
  { $unwind: "$items" },
  { $group: { _id: null,
    total_pedidos: { $sum: 1 },
    facturacion_total: { $sum: { $multiply: ["$items.cantidad", "$items.precio"] } },
    ticket_promedio: { $avg: { $multiply: ["$items.cantidad", "$items.precio"] } }
  } }
])

```

```
    } }  
  ])
```

Paso 12 — Preguntar a la IA

Mostrame la diferencia entre usar `$group { _id: "$cliente_id" }` y `{ _id: null }`.

¿Cómo haría para quedarme solo con los 2 clientes que más gastaron?

¿Qué ventaja tiene `$lookup` frente a guardar los datos anidados? ¿Cuándo conviene cada uno?

Verificación de resultados

- ☐ El paso 7 muestra el orden correcto de clientes por total gastado.
- ☐ El paso 8 muestra "Monitor" con 2 unidades y más ingresos.
- ☐ El paso 9 agrupa por mes (enero, febrero, marzo).
- ☐ El paso 10 devuelve el nombre del cliente dentro de cada pedido.

Criterios de evaluación

Criterio	Puntos
Datos cargados correctamente	15
<code>\$match</code> + <code>\$unwind</code> correctos	20
<code>\$group</code> con <i>sum</i> /avg correcto	25
<code>\$lookup</code> con la unión correcta	20
Consulta 11 (resumen general) y explicación	20

Entregable

- Archivo `agregacion-mongodb.js` con todos los pipelines y el resultado de cada uno.
- Una respuesta de la IA (captura) sobre la pregunta de `_id: null`.